Université Hassan 1er

ÉCOLE NATIONALE DE COMMERCE ET DE GESTION DE SETTAT

RAPPORT DE PROJET

Détection de Fraudes sur les Transactions par Carte Bancaire au Maroc

Module :	Intelligence Artificielle
Filière :	Finance
Réalisé par :	Rim Hammani / Chakir Malika Malak
Encadrant :	Abderrahim Larhlimi
Période :	Année Universitaire 2024–2025

Année Universitaire 2024–2025

Sommaire

I. Explication du Jeu de Données 3
I.1 Description des Variables 3
I.2 Prétraitement Nécessaire 4
II. Code Python de Régression Logistique 5
III. Résultats et Interprétation 6
III.1 Matrice de Confusion 6
III.2 Rapport de Classification et Accuracy 7
III.3 Conclusion et Pistes d'Amélioration 8

I. Explication du Jeu de Données

Le jeu de données analysé, intitulé maroc_bank_transactions.csv, contient un historique de transactions financières composé de 500 observations enregistrées par plusieurs banques marocaines. Son objectif principal est de permettre la création de modèles prédictifs — tels que la régression logistique — afin de distinguer les transactions légitimes des transactions frauduleuses.

I.1 Description des Variables (Caractéristiques)

Le dataset est composé des colonnes suivantes :

Variable	Type	Description
bank_name	Catégorielle	Banque émettrice de la carte (BMCE Bank, CIH Bank, Crédit du Maroc, Banque Populaire, Attijariwafa Bank).
city	Catégorielle	Ville où la transaction a eu lieu (Casablanca, Marrakech, Fès, Tanger, Agadir, Rabat).
transaction_id	Identifiant	Identifiant unique de chaque transaction. Non pertinent pour la modélisation.
card_type	Catégorielle	Réseau de la carte bancaire utilisée (Mastercard, Visa, AMEX).
amount_MAD	Numérique continue	Montant de la transaction en Dirhams Marocains (MAD). Seule variable numérique continue.
time	Date/Heure	Horodatage exact de la transaction.
type_fraud	Catégorielle	Méthode de fraude utilisée (Skimming, Online, Clonage, Phishing) ou 'None' si légitime.
status	Binaire (Cible)	Variable cible y : 0 = transaction légitime ; 1 = transaction frauduleuse.

I.2 Prétraitement Nécessaire

Afin d'appliquer un modèle de régression logistique dans de bonnes conditions, une préparation rigoureuse des données s'avère indispensable. Les étapes suivantes ont été identifiées :

- Exclusion des variables non pertinentes : transaction_id (aucune valeur informative), time (à moins d'en extraire l'heure ou le jour de la semaine), et surtout type_fraud — cette variable constitue une fuite de données (data leakage) car elle n'est connue qu'après qualification de la transaction.

- Encodage des variables catégorielles : transformation de bank_name, city et card_type en valeurs numériques par la méthode One-Hot Encoding.
- Mise à l'échelle (Feature Scaling) : standardisation du montant amount_MAD, la régression logistique étant sensible à l'échelle des variables numériques.
- Gestion du déséquilibre de classes : les transactions frauduleuses représentent une minorité des observations. Le recours au paramètre class_weight='balanced' est fortement recommandé pour corriger ce biais.

II. Code Python de Régression Logistique (Scikit-Learn)

Le script suivant, enregistré sous le nom train_lr.py, constitue le pipeline complet d'entraînement et d'évaluation du modèle de régression logistique sur le jeu de données maroc_bank_transactions.csv. Chaque étape est commentée afin d'en faciliter la compréhension et la reproductibilité.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix

# 1. Chargement du jeu de données
df = pd.read_csv('maroc_bank_transactions.csv')

# 2. Nettoyage et sélection des variables
df_clean = df.drop(columns=['transaction_id', 'time', 'type_fraud'])
X = df_clean.drop(columns=['status'])
y = df_clean['status']

# 3. Identification des types de variables
categorical_cols = ['bank_name', 'city', 'card_type']
numerical_cols   = ['amount_MAD']

preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numerical_cols),
    ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
categorical_cols)
])
X_preprocessed = preprocessor.fit_transform(X)

# 4. Découpage Train/Test (80%/20%) avec stratification
X_train, X_test, y_train, y_test = train_test_split(
    X_preprocessed, y, test_size=0.2, random_state=42, stratify=y)

# 5. Entraînement du modèle avec pondération des classes
model = LogisticRegression(class_weight='balanced', random_state=42)
model.fit(X_train, y_train)

# 6. Prédictions et évaluation
y_pred = model.predict(X_test)
print(f"Accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Ce pipeline respecte les bonnes pratiques en matière de modélisation : séparation stricte des ensembles d'entraînement et de test, gestion du déséquilibre par pondération, et encodage adapté à chaque type de variable.

III. Résultats et Interprétation

Les résultats présentés ci-après ont été obtenus suite à l'exécution du script train_lr.py sur l'ensemble de test, représentant 20 % des 500 transactions initiales, soit un échantillon de 100 observations.

```
Accuracy: 0.67

Confusion Matrix:
 [[64 29]
  [ 4  3]]

Classification Report:
              precision    recall  f1-score   support
           0       0.94      0.69      0.80        93
           1       0.09      0.43      0.15         7
    accuracy                           0.67       100
   macro avg       0.52      0.56      0.47       100
weighted avg       0.88      0.67      0.75       100
```

III.1 Matrice de Confusion

La matrice de confusion détaille la répartition des 100 prédictions entre les cas correctement et incorrectement classés par le modèle. Son analyse permet de dépasser l'indicateur global d'accuracy, souvent trompeur en contexte déséquilibré.

	Prédit : Non-Fraude (0)	**Prédit : Fraude (1)**
Réel : Non-Fraude (0)	64 — Vrais Négatifs (VN) ✓	29 — Faux Positifs (FP) ✗
Réel : Fraude (1)	4 — Faux Négatifs (FN) ✗	3 — Vrais Positifs (VP) ✓

- 64 Vrais Négatifs (VN) : le modèle a correctement identifié 64 transactions légitimes.
- 29 Faux Positifs (FP) : 29 transactions normales ont été flaggées à tort comme frauduleuses, générant des fausses alertes susceptibles d'entraîner des blocages abusifs de cartes.
- 4 Faux Négatifs (FN) : 4 transactions frauduleuses ont été laissées passer. Il s'agit du type d'erreur le plus préjudiciable sur le plan financier et de la confiance client.
- 3 Vrais Positifs (VP) : le modèle a correctement intercepté 3 transactions frauduleuses.

III.2 Rapport de Classification et Précision Globale

L'analyse du rapport de classification révèle plusieurs enseignements importants sur les performances réelles du modèle :

Accuracy globale (0,67 soit 67 %)

Le modèle a produit une prédiction correcte pour 67 transactions sur 100. Ce chiffre est néanmoins trompeur : un modèle naïf prédisant systématiquement « Non-Fraude » aurait atteint une accuracy de 93 %, sans détecter la moindre fraude. Ce phénomène, connu sous le nom de paradoxe de la précision, est inhérent aux jeux de données déséquilibrés.

Rappel (Recall) pour la classe Fraude = 0,43 (43 %)

Sur les 7 transactions frauduleuses réelles présentes dans l'échantillon de test, le modèle en a détecté 3, soit 43 %. Ce résultat, bien qu'insuffisant d'un point de vue opérationnel (57 % des fraudes passent inaperçues), représente une amélioration notable par rapport à un modèle non pondéré, grâce à l'utilisation de class_weight='balanced'.

Précision (Precision) pour la classe Fraude = 0,09 (9 %)

Sur les 32 alertes à la fraude déclenchées par le modèle (3 VP + 29 FP), celui-ci n'a raison que dans 9 % des cas. Autrement dit, 91 % des alertes sont de fausses alarmes. Ce niveau de précision très bas implique une charge investigatrice importante pour les équipes de conformité et de gestion des risques.

III.3 Conclusion et Pistes d'Amélioration

Le modèle de régression logistique, bien qu'il intègre un mécanisme de pondération pour cibler les transactions frauduleuses, peine à établir une frontière de décision claire sur la base des variables disponibles (banque, ville, type de carte, montant). Ses hypothèses de linéarité ne sont pas suffisantes pour discriminer efficacement les comportements frauduleux, qui présentent souvent des patterns complexes et non linéaires.

Pistes d'amélioration identifiées :

- Changement d'algorithme : le recours à des modèles non linéaires tels que Random Forest ou XGBoost permettrait de capter des interactions plus complexes entre variables et d'améliorer significativement les métriques de détection.

- Feature Engineering temporel : l'extraction de caractéristiques depuis la variable time (heure, jour de la semaine, week-end/jour ouvré) apporterait une information discriminante, les comportements frauduleux présentant généralement des patterns temporels distincts.
- Techniques d'échantillonnage : l'application de méthodes SMOTE (Synthetic Minority Over-sampling Technique) ou d'under-sampling sur la classe majoritaire constituerait une alternative à la simple pondération des classes.